In [ ]:
```
# If needed, run once:
#%pip install -q pandas scikit-learn tensorflow ann_visualizer graphviz matplotlib
```

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

TARGET_COLUMN = "Exited"               # Change this to your label column

df = pd.read_csv(r"E:\BE\Practical\ML_Practical\Practical_3\Churn_Modelling.csv")
print(df.shape)
display(df.head())
print(df.dtypes)
```

```
(10000, 14)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Bal
0	1	15634602	Hargrave	619	France	Female	42	2	
1	2	15647311	Hill	608	Spain	Female	41	1	838
2	3	15619304	Onio	502	France	Female	42	8	1596
3	4	15701354	Boni	699	France	Female	39	1	
4	5	15737888	Mitchell	850	Spain	Female	43	2	1255

```
RowNumber            int64
CustomerId           int64
Surname             object
CreditScore          int64
Geography           object
Gender              object
Age                  int64
Tenure               int64
Balance            float64
NumOfProducts        int64
HasCrCard            int64
IsActiveMember       int64
EstimatedSalary    float64
Exited               int64
dtype: object
```

In [2]:
```
df = df.dropna(subset=[TARGET_COLUMN]).copy()

X = df.drop(
    columns=["Exited", "RowNumber", "CustomerId", "Surname"],
    errors="ignore"
)

y = df[TARGET_COLUMN].astype(str)

numeric_cols = X.select_dtypes(include=np.number).columns.tolist()
categorical_cols = X.select_dtypes(exclude=np.number).columns.tolist()

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), numeric_cols),

    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_cols)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

if hasattr(X_train, "toarray"):
    X_train = X_train.toarray()
    X_test = X_test.toarray()

label_encoder = LabelEncoder()
y_train_enc = label_encoder.fit_transform(y_train)
y_test_enc = label_encoder.transform(y_test)

n_classes = len(label_encoder.classes_)

print("Classes:", list(label_encoder.classes_))
print("Input features after preprocessing:", X_train.shape[1])
```

```
Classes: ['0', '1']
Input features after preprocessing: 13
```

In [3]:
```
model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(n_classes, activation="softmax")
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

```
model.summary()

history = model.fit(X_train, y_train_enc, validation_split=0.2,
                    epochs=15, batch_size=32, verbose=1)
loss, acc = model.evaluate(X_test, y_test_enc, verbose=0)
pred = np.argmax(model.predict(X_test, verbose=0), axis=1)
print(f"Test accuracy: {acc:.4f}")
print(classification_report(y_test_enc, pred, target_names=label_encoder.classes_))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 32)	
dense_1 (Dense)	(None, 16)	
dense_2 (Dense)	(None, 2)	

Total params: 1,010 (3.95 KB)

Trainable params: 1,010 (3.95 KB)

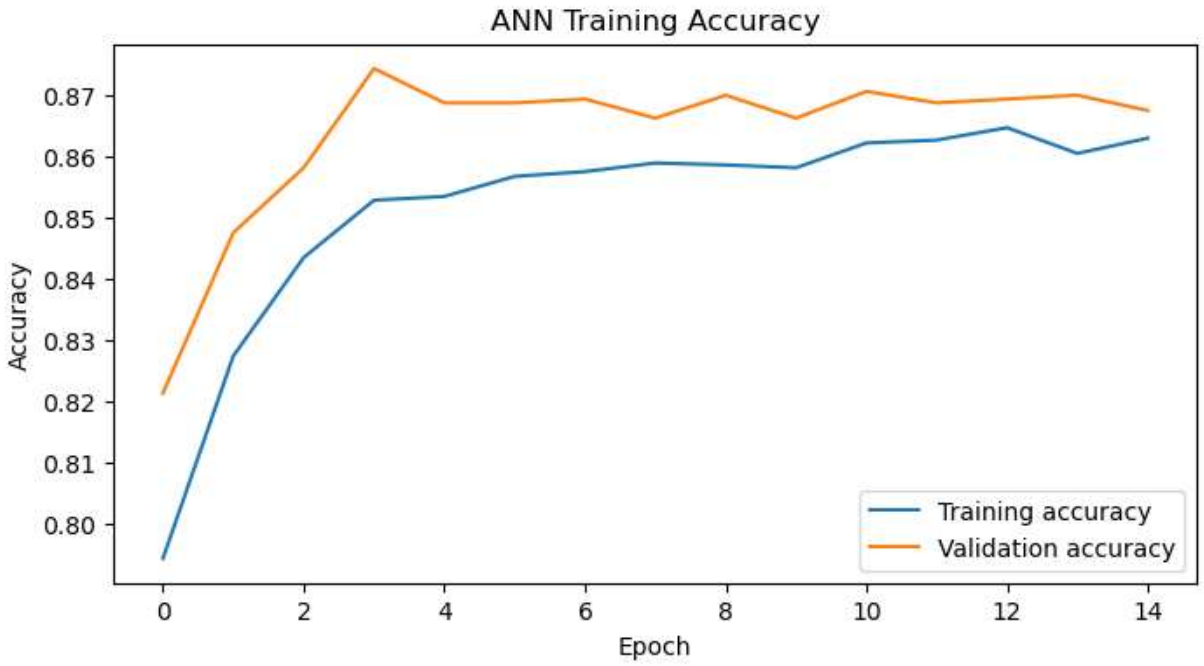
Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7942 - loss: 0.4693 - val_acc
uracy: 0.8213 - val_loss: 0.4240
Epoch 2/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8273 - loss: 0.4060 - val_acc
uracy: 0.8475 - val_loss: 0.3907
Epoch 3/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8434 - loss: 0.3812 - val_acc
uracy: 0.8581 - val_loss: 0.3636
Epoch 4/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 993us/step - accuracy: 0.8528 - loss: 0.3618 - val_a
ccuracy: 0.8744 - val_loss: 0.3475
Epoch 5/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 986us/step - accuracy: 0.8534 - loss: 0.3522 - val_a
ccuracy: 0.8687 - val_loss: 0.3387
Epoch 6/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 988us/step - accuracy: 0.8567 - loss: 0.3484 - val_a
ccuracy: 0.8687 - val_loss: 0.3356
Epoch 7/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8575 - loss: 0.3450 - val_acc
uracy: 0.8694 - val_loss: 0.3377
Epoch 8/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8589 - loss: 0.3449 - val_acc
uracy: 0.8662 - val_loss: 0.3333
Epoch 9/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8586 - loss: 0.3412 - val_acc
uracy: 0.8700 - val_loss: 0.3320
Epoch 10/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8581 - loss: 0.3400 - val_acc
uracy: 0.8662 - val_loss: 0.3333
Epoch 11/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8622 - loss: 0.3382 - val_acc
uracy: 0.8706 - val_loss: 0.3339
Epoch 12/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8627 - loss: 0.3366 - val_acc
uracy: 0.8687 - val_loss: 0.3332
Epoch 13/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 986us/step - accuracy: 0.8647 - loss: 0.3351 - val_a
ccuracy: 0.8694 - val_loss: 0.3348
Epoch 14/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8605 - loss: 0.3345 - val_acc
uracy: 0.8700 - val_loss: 0.3324
Epoch 15/15
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8630 - loss: 0.3321 - val_acc
uracy: 0.8675 - val_loss: 0.3341
Test accuracy: 0.8600
              precision    recall  f1-score   support

           0       0.87      0.97      0.92      1593
           1       0.79      0.43      0.55       407

    accuracy                           0.86      2000
   macro avg       0.83      0.70      0.74      2000
weighted avg       0.85      0.86      0.84      2000
```

In [4]:
```
plt.figure(figsize=(8, 4))
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("ANN Training Accuracy")
plt.show()
```



In [6]:
```
#%pip install ann_visualizer graphviz
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting ann_visualizer
  Downloading ann_visualizer-2.5.tar.gz (4.7 kB)
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Getting requirements to build wheel: started
  Getting requirements to build wheel: finished with status 'done'
  Preparing metadata (pyproject.toml): started
  Preparing metadata (pyproject.toml): finished with status 'done'
Collecting graphviz
  Downloading graphviz-0.21-py3-none-any.whl.metadata (12 kB)
Downloading graphviz-0.21-py3-none-any.whl (47 kB)
Building wheels for collected packages: ann_visualizer
  Building wheel for ann_visualizer (pyproject.toml): started
  Building wheel for ann_visualizer (pyproject.toml): finished with status 'done'
  Created wheel for ann_visualizer: filename=ann_visualizer-2.5-py3-none-any.whl siz
e=4236 sha256=650317162d4bd15f33ce15f6341761674de22dd85db5ce4686db899e4c443ca3
  Stored in directory: c:\users\om_jangam\appdata\local\pip\cache\wheels\95\85\5d\0b
9fbe0b04b6753e40d2c5de722ac7938c661fe23f7e83c32d
Successfully built ann_visualizer
Installing collected packages: ann_visualizer, graphviz

   ---------------------------------------- 2/2 [graphviz]

Successfully installed ann_visualizer-2.5 graphviz-0.21
Note: you may need to restart the kernel to use updated packages.
```

In [8]:
```
#%pip install pydot graphviz
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting pydot
  Downloading pydot-4.0.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: graphviz in c:\users\om_jangam\appdata\roaming\python
\python313\site-packages (0.21)
Requirement already satisfied: pyparsing>=3.1.0 in c:\programdata\anaconda3\lib\site
-packages (from pydot) (3.2.5)
Downloading pydot-4.0.1-py3-none-any.whl (37 kB)
Installing collected packages: pydot
Successfully installed pydot-4.0.1
Note: you may need to restart the kernel to use updated packages.
```

In [5]:
```
# ANN Architecture Visualization using Matplotlib
import matplotlib.pyplot as plt
from matplotlib.patches import Circle

# Get the number of neurons in each Dense layer
input_size = model.input_shape[-1]

if input_size is None:
    input_size = model.layers[0].input.shape[-1]

layers = [int(input_size)]
for layer in model.layers:
    if hasattr(layer, "units"):
        layers.append(layer.units)
```

```
fig, ax = plt.subplots(figsize=(12, 7))

max_display = 8
layer_positions = []

# Calculate neuron positions
for i, count in enumerate(layers):
    display_count = min(count, max_display)
    positions = []

    for j in range(display_count):
        y = (display_count - 1) / 2 - j
        positions.append((i * 2.5, y))

    layer_positions.append(positions)

# Draw connections
for i in range(len(layer_positions) - 1):
    for x1, y1 in layer_positions[i]:
        for x2, y2 in layer_positions[i + 1]:
            ax.plot([x1, x2], [y1, y2],
                    color="gray", linewidth=0.5, alpha=0.25)

# Draw neurons and labels
for i, positions in enumerate(layer_positions):
    for x, y in positions:
        ax.add_patch(Circle((x, y), 0.13,
                            facecolor="skyblue",
                            edgecolor="black",
                            zorder=3))

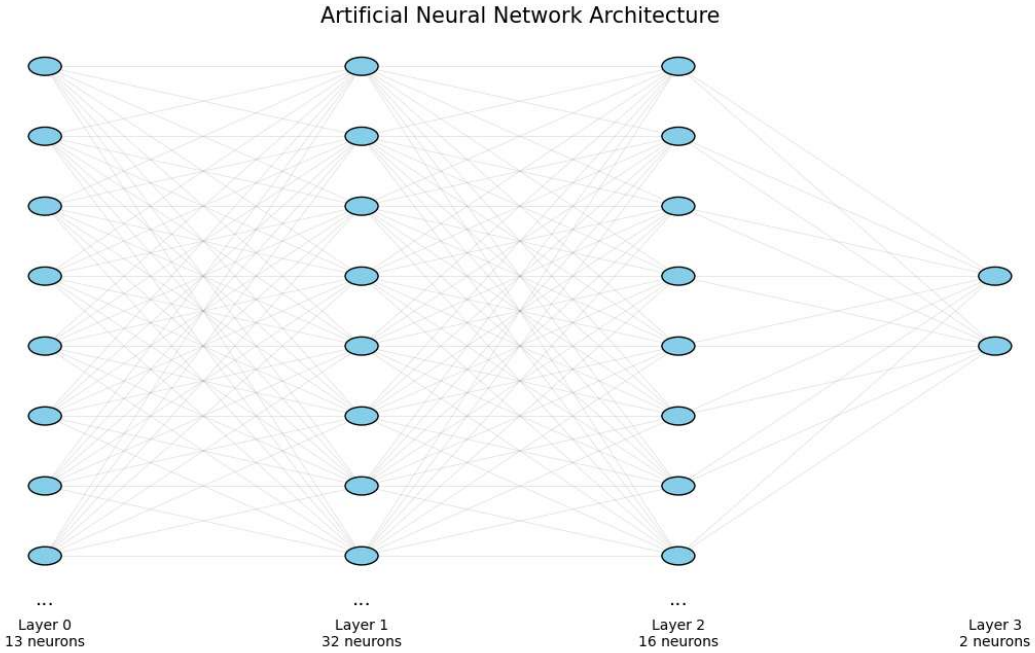
    count = layers[i]
    if count > max_display:
        ax.text(i * 2.5, -max_display / 2 - 0.2,
                "...", ha="center", fontsize=14)

    ax.text(i * 2.5, -max_display / 2 - 0.8,
            f"Layer {i}\n{count} neurons",
            ha="center", fontsize=10)

ax.set_title("Artificial Neural Network Architecture", fontsize=15)
ax.set_xlim(-1, (len(layers) - 1) * 2.5 + 1)
ax.set_ylim(-max_display / 2 - 1.5, max_display / 2)
ax.axis("off")

plt.tight_layout()
plt.savefig("ann_architecture.png", dpi=200, bbox_inches="tight")
plt.show()

print("Architecture diagram saved as ann_architecture.png")
```




```
Architecture diagram saved as ann_architecture.png
```

In [ ]: